

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Last Week's Lecture

React Refresher

- Declarative UI updates for interactive apps
- Components, props, hooks (`useState`, `useEffect`)

React Native: Cross-platform native apps with one JavaScript codebase

Expo: Simplifies setup, testing, and sensor access

- Workflow: Initialize, run, and preview with hot reloading

Today's Lecture

React Native Fundamentals

- Core concepts, components, and layout
- State, props, and basic interactivity

By the end, you'll be ready to complete Assignment 1 on your own

Assignment 1

Build the foundational structure of the Fitness Tracker App

- **Log** activities with details (type, duration, calories)
- **Display** activities in a list
- **Update** activities from the list
- **Delete** activities from the list
- **Input validation** and **state management**

React Native

React Native lets you build **native mobile apps** using JavaScript and React

- Use the same **component-based** approach as React
- Write **one codebase** for both iOS and Android
- JavaScript and React components are translated into **native UI elements**

Key Difference from React:

- No HTML elements (e.g., `<div>`, `<button>`)
- Use React Native components (e.g., `<View>`, `<Button>`)

React Native Components

React Native provides built-in **Core Components** for building UI

Docs: reactnative.dev/docs/components-and-apis

- `<View>`: Container for other components (like `<div>` in web)
- `<Text>`: Displays text
- `<Button>`: Basic button with a title and `onPress` event

Example: Simple UI

```
1 import { Button, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View>
6       <Text>Welcome to React Native!</Text>
7       <Button title="Click Me" onPress={() => alert("Button pressed!")} /
8     </View>
9   );
10 }
```

- `<View>` acts as a container
- `<Text>` displays static text
- `<Button>` triggers an alert when pressed

Live Demo

Initialize project:

```
npx create-expo-app@latest
```

- `--template` option
- `default`: Designed to build multi-screen apps
- `blank-typescript`: A Blank template with TypeScript enabled

```
npx create-expo-app@latest --template blank-typescript
```

Run:

```
npx expo start
```


Live Demo

Edit `App.tsx`

my-app/App.tsx

```
1 import { Button, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View>
6       <Text>Welcome to React Native!</Text>
7       <Button title="Click Me" onPress={() => alert("Button pressed!")} /
8     </View>
9   );
10 }
```

- Default styling behavior: Components in `<View>` are positioned at the top of the screen

Styling in React Native

React Native uses **JavaScript objects** for styling, not CSS

- Apply styles with the `style` prop

my-app/App.tsx

```
1 import { Button, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View style={{ flex: 1, justifyContent: "center", alignItems: "center"
6       <Text>Welcome to React Native!</Text>
7       <Button title="Click Me" onPress={() => alert("Button pressed!")} /
8     </View>
9   );
10 }
```

Layout in React Native

React Native uses **Flexbox** for positioning elements

Flexbox: Layout system that arranges components in a row or a column and distributes space between them

Flexbox in React Native

Default `flexDirection`: "column" (children stack vertically)

Common properties:

- `flex`: Relative size compared to siblings (`flex: 1`)
- `justifyContent`: How to align children along main axis
- `flexDirection`: "column": Main axis = Vertical
- `flexDirection`: "row": Main axis = Horizontal
- `alignItems`: How to align children along cross axis

Doc: reactnative.dev/docs/flexbox

Styling: Example

```
<View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
  <Text>Welcome to React Native!</Text>
  <Button title="Click Me" onPress={() => alert("Button pressed!")} />
</View>
```

- `flex: 1`: Expand `<View>` to fill the entire available space of its parent (e.g., the screen)
- `justifyContent: "center"`: Center child components vertically
- `alignItems: "center"`: Center child components (e.g., `<Text>`, `<Button>`) horizontally

Styling

Use `StyleSheet` for better organization

- A React Native component; provides an abstraction similar to CSS StyleSheets

my-app/App.tsx

```
1 import { Button, StyleSheet, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View style={styles.container}>
6       <Text>Welcome to React Native!</Text>
7       <Button title="Click Me" onPress={() => alert("Button pressed!")} /
8     </View>
9   );
10 }
11
12 const styles = StyleSheet.create({
13   container: {
14     flex: 1,
15     backgroundColor: "#d6f5d6",
16     alignItems: "center",
17     justifyContent: "center",
18   },
19 });
```

StyleSheet: Example

my-app/App.tsx

```
1 import { Button, StyleSheet, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View style={styles.container}>
6       <Text style={styles.text}>Welcome to React Native!</Text>
7       <Button title="Click Me" onPress={() => alert("Button pressed!")} /
8     </View>
9   );
10 }
11
12 const styles = StyleSheet.create({
13   container: {
14     flex: 1,
15     backgroundColor: "#d6f5d6",
16     alignItems: "center",
17     justifyContent: "center",
18   },
19   text: {
```

Optimized Rendering

`StyleSheet.create` generates a unique ID for each style object and sends it to the native layer once, where it's cached

```
1 const styles = StyleSheet.create({
2   container: {
3     flex: 1,
4     backgroundColor: "#d6f5d6",
5     alignItems: "center",
6     justifyContent: "center",
7   },
8 });
```

- `styles.container` is created once, cached, and reused
- Reduce the amount of data sent across the React Native bridge (from JavaScript to native) during rendering

Optimized Rendering

StyleSheet.create: Generates a unique ID for each style object and sends it to the native layer once, where it's cached

- Reduce the amount of data sent across the React Native bridge (from JavaScript to native) during rendering

Inline Styles: Serialized and sent across the bridge **every time** a component renders

Recap: Styling

React Native uses **JavaScript objects** for styling

- Apply styles with the `style` prop
- E.g., `<View style={styles.container}> </View>`
- Use `StyleSheet` for better organization and performance

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
  },
});
```

- Use **Flexbox** for positioning elements

React Native Components

- **View**: Container for other components
- **Text**: Displays text
- **Button**: Basic button
- **StyleSheet**: Abstraction similar to CSS StyleSheets
- **TextInput**: Input field for user text
- Assignment 1: Log activities with details (type, duration, calories)

TextInput

A foundational component for inputting text into the app via a keyboard

```
<TextInput
  placeholder="Enter a positive integer"
  value={integer}
  onChangeText={setInteger}
  keyboardType="numeric"
/>
```

- **placeholder**: Placeholder text
- **value**: Current text value (string)
- **onChangeText**: Function called when text changes
- **keyboardType**: Keyboard type, e.g., “numeric” shows number pad (not strict validation)

TextInput: Example

```
<TextInput
  placeholder="Enter a positive integer"
  value={integer}
  onChangeText={setInteger}
  keyboardType="numeric"
/>
```

- **value**: Current text value (string)
 - **onChangeText**: Function called when text changes
- ```
const [integer, setInteger] = useState("");
```

# Handle User Input

- Use `<TextInput>` for text input
- Use `<Button>` for actions
- `onPress` prop: Handles button press

Example:

- User types text into the input field (`<TextInput>`)
- User presses the button (`<Button>`)
- An alert pops up showing: “You entered: `<typed text>`”
- Typed text needs to be stored in state

# Live Coding

User types text into the input field (`<TextInput>`)

User presses the button (`<Button>`)

An alert pops up: “You entered: `<typed text>`” (`useState`)

my-app/App.tsx

```
1 import { View, TextInput, Button } from "react-native";
2
3 export default function App() {
4 return (
5 <View>
6 <TextInput />
7 <Button />
8 </View>
9);
10 }
```

# Example: User Input

my-app/App.tsx

```
1 import { useState } from "react";
2 import { View, TextInput, Button } from "react-native";
3
4 export default function App() {
5 const [text, setText] = useState("");
6
7 return (
8 <View>
9 <TextInput placeholder="Enter text" value={text} onChangeText={setT
10 <Button title="Submit" onPress={() => alert(`You entered: ${text}`)}
11 </View>
12);
13 }
```

- `useState` manages input state
- `<TextInput>` updates `text` on change
- `<Button>` shows an alert with the input



# Live Demo

my-app/App.tsx

```
1 import { useState } from "react";
2 import { View, TextInput, Button, StyleSheet } from "react-native";
3
4 export default function App() {
5 const [text, setText] = useState("");
6
7 return (
8 <View style={styles.container}>
9 <TextInput
10 style={styles.input}
11 placeholder="Enter text"
12 value={text}
13 onChangeText={setText}
14 />
15 <Button title="Submit" onPress={() => alert(`You entered: ${text}`)}
16 </View>
17);
18 }
19
```

- Assignment 1: Log activities with details (type, duration, calories)

# React Native Components

**View**: Container for other components

**Text**: Displays text

**Button**: Basic button

**TextInput**: Input field for user text

**ScrollView**: Scrollable container for multiple components and views

# View: Limitation

Content cannot scroll

- Items outside the screen are cut off

my-app/App.tsx

```
1 import { View, Text } from "react-native";
2
3 export default function App() {
4 return (
5 <View>
6 {Array.from({ length: 20 }, (_, i) => (
7 <Text key={i} style={{ fontSize: 40 }}>
8 Item {i + 1}
9 </Text>
10))}
11 </View>
12);
13 }
```

# ScrollView

Enable scrolling when content doesn't fit on screen

my-app/App.tsx

```
1 import { ScrollView, Text } from "react-native";
2
3 export default function App() {
4 return (
5 <ScrollView>
6 {Array.from({ length: 20 }, (_, i) => (
7 <Text key={i} style={{ fontSize: 40 }}>
8 Item {i + 1}
9 </Text>
10))}
11 </ScrollView>
12);
13 }
```

Documentation: [reactnative.dev/docs/scrollview](https://reactnative.dev/docs/scrollview)

## ScrollView: Performance

Render all children at once

- Good for small, static sets of items
- Not efficient for long, dynamic lists (e.g. large dataset)

Solution **FlatList**

- Render only visible items
- Built-in props for scrolling, refreshing, and virtualization
- Handle large or dynamic datasets efficiently

# FlatList

Efficiently render a scrollable list of items

- Key props:
  - `data`: Array of items to render
  - `renderItem`: Function to render each item
  - `keyExtractor`: Function to return a unique key

```
const items = [{ id: "1", name: "Item 1" }];

<FlatList
 data={items}
 keyExtractor={(item) => item.id}
 renderItem={({ item }) => <Text>{item.name}</Text>}
/>
```

# FlatList: Example

## Display a list of items

my-app/App.tsx

```
1 import { View, Text, FlatList } from "react-native";
2
3 const items = [
4 { id: "1", name: "Item 1" },
5 { id: "2", name: "Item 2" },
6];
7
8 export default function App() {
9 return (
10 <View>
11 <FlatList
12 data={items}
13 keyExtractor={(item) => item.id}
14 renderItem={({ item }) => <Text>{item.name}</Text>}
15 />
16 </View>
17);
18 };
```

# Live Demo

my-app/App.tsx

```
1 import { View, Text, FlatList, StyleSheet } from "react-native";
2
3 const items = [
4 { id: "1", name: "Item 1" },
5 { id: "2", name: "Item 2" },
6];
7
8 export default function App() {
9 return (
10 <View style={styles.container}>
11 <FlatList
12 data={items}
13 keyExtractor={(item) => item.id}
14 renderItem={({ item }) => <Text style={styles.item}>{item.name}</
15 />
16 </View>
17);
18 }
19
```



# FlatList with Flexbox

Why `justifyContent: "center"` did not work?

- `<FlatList>` is the only child of `<View>`
- `<FlatList>` expands to fill all available vertical space
- So centering the child `<FlatList>` vertically has no visible effect

To center items in `Flatlist`

- Use `contentContainerStyle` prop to style list items

# contentContainerStyle

```
<FlatList
 contentContainerStyle={styles.list}
 data={items}
 keyExtractor={({item}) => item.id}
 renderItem={({ item }) => <Text style={styles.item}>{item.name}</Text>}
/>
```

## With StyleSheet

```
const styles = StyleSheet.create({
 list: {
 flex: 1,
 justifyContent: "center",
 },
});
```

# Live Demo

my-app/App.tsx

```
1 import { View, Text, FlatList, StyleSheet } from "react-native";
2
3 const items = [
4 { id: "1", name: "Item 1" },
5 { id: "2", name: "Item 2" },
6];
7
8 export default function App() {
9 return (
10 <View style={styles.container}>
11 <FlatList
12 contentContainerStyle={styles.list}
13 data={items}
14 keyExtractor={(item) => item.id}
15 renderItem={({ item }) => <Text style={styles.item}>{item.name}</Text> }
16 />
17 </View>
18);
19 }
```

## FlatList: Performance

Only render items currently visible on screen

Automatically reuse components as you scroll

- Improve memory usage and scrolling performance
- Useful for large or dynamic datasets

# FlatList: Performance

Configurable props:

- `initialNumToRender`: How many items to render initially
- `maxToRenderPerBatch`: How many items to render per scroll batch
- `windowSize`: How many screens worth of items to keep in memory

Doc: [Optimizing Flatlist Configuration](#)

## Recap: `ScrollView` vs. `FlatList`

`ScrollView`: Renders **all children at once**

- Good for **small, static** lists

`FlatList`: Renders **only visible items** (+ buffer)

- Designed for **large, dynamic** lists
- Provides extra props for performance & layout control

Assignment 1: Display a list of logged activities

# Add Interactivity to List

Combine `<FlatList>` with `<Button>` to make lists interactive

Assignment 1:

- Delete activities from the list
- Update activities from the list

# Add Interactivity to List

Combine `<FlatList>` with `<Button>` to make lists interactive

Example: Delete an item from a list

- Use `<FlatList>` to display a list of items
- Each item is rendered with a **Delete button** (`<Button>`)
- When user presses **Delete**, remove the corresponding item from list
- To automatically update UI, manage items in **component state**



# Live Coding

my-app/App.tsx

```
1 import { View, Text, FlatList } from "react-native";
2
3 const items = [
4 { id: "1", name: "Item 1" },
5 { id: "2", name: "Item 2" },
6];
7
8 export default function App() {
9 return (
10 <View>
11 <FlatList
12 data={items}
13 keyExtractor={(item) => item.id}
14 renderItem={({ item }) => <Text>{item.name}</Text>}
15 />
16 </View>
17);
18 }
```

# Add Interactivity to List

my-app/App.tsx

```
1 import { useState } from "react";
2 import { View, Text, FlatList, Button } from "react-native";
3
4 export default function App() {
5 const [items, setItems] = useState([
6 { id: "1", name: "Item 1" },
7 { id: "2", name: "Item 2" },
8]);
9
10 const deleteItem = (id) => {
11 setItems(items.filter((item) => item.id !== id));
12 };
13
14 return (
15 <View>
16 <FlatList
17 data={items}
18 keyExtractor={(item) => item.id}
19 renderItem={({ item }) => (
```

useState manages the list

deleteItem filters out the item by id

# TypeScript

## Missing type annotation

```
const deleteItem = (id) => {
 setItems(items.filter((item) => item.id !== id));
};
```

Parameter 'id' implicitly has an 'any' type.

## Add type annotation

```
const deleteItem = (id: string) => {
 setItems(items.filter((item) => item.id !== id));
};
```

- TypeScript checks that `id` is a string
- Prevents accidental type errors during development

# Type Safety

TypeScript adds **type safety** to React Native apps

- Define types for state, props, and data
- Catch errors during development
- Improve code readability and maintainability

# Type State

```
const [items, setItems] = useState([
 { id: "1", name: "Item 1" },
 { id: "2", name: "Item 2" },
]);
```

- If the initial state is an empty array, TypeScript infers `any[]`
- But with given objects, TypeScript infers `{ id: string; name: string }[]`
- Better: define a named type for clarity

```
type Item = {
 id: string;
 name: string;
};
```

# Type State

Define a proper `Item` type

```
type Item = {
 id: string;
 name: string;
};
```

Use `useState<Item[]>` to type state

```
const [items, setItems] = useState<Item[]>([
 { id: "1", name: "Item 1" },
 { id: "2", name: "Item 2" },
]);
```

- Ensure `items` is always an array of `Item`
- Ensure `setItems` accepts only valid `Item[]`

# Organize Your Types

Create a separate file for types:

- Easier to share types across multiple components
- Keep code modular and maintainable

my-app/types.ts

```
1 export type Item = {
2 id: string;
3 name: string;
4 };
```

Use it in other files:

my-app/App.tsx

```
import { Item } from "../types";
```

# Live Demo

my-app/App.tsx

```
1 import { useState } from "react";
2 import { View, Text, FlatList, Button, StyleSheet } from "react-native";
3 import { Item } from "../types";
4
5 export default function App() {
6 const [items, setItems] = useState<Item[]>([
7 { id: "1", name: "Item 1" },
8 { id: "2", name: "Item 2" },
9]);
10
11 const deleteItem = (id: Item["id"]) => {
12 setItems(items.filter((item) => item.id !== id));
13 };
14
15 return (
16 <View style={styles.container}>
17 <FlatList
18 data={items}
19 keyExtractor={(item) => item.id}
```



## **paddingTop: 100**

Avoid content being blocked by status bar, notches

- Works, but not elegant or adaptive

Better approach: **SafeAreaView**

# SafeAreaView

From `react-native`

- `import { SafeAreaView } from "react-native";`
- iOS React Native Component: Render content within the safe area boundaries of a iOS device
- Deprecated

From `react-native-safe-area-context`

- `import { SafeAreaView } from "react-native-safe-area-context";`
- Work for both iOS and Android

# Safe Area: Set Up

Install `react-native-safe-area-context`

---

```
npm install react-native-safe-area-context
```

---

# Safe Area: Components

## [react-native-safe-area-context](#) Documentation

- **SafeAreaProvider**: Should be added in app root component

```
export default function App() {
 return <SafeAreaProvider>{ /*...*/ }</SafeAreaProvider>;
}
```

- **SafeAreaView**: Regular **View** component with the safe area insets applied as padding or margin

# Safe Area: Example

---

```
1 import { Text } from "react-native";
2 import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
3
4 export default function App() {
5 return (
6 <SafeAreaProvider>
7 <SafeAreaView>
8 <Text>Page content</Text>
9 </SafeAreaView>
10 </SafeAreaProvider>
11);
12 }
```

# Live Coding

## Restyle previous demo

my-app/App.tsx

```
1 import { useState } from "react";
2 import { View, Text, FlatList, Button, StyleSheet } from "react-native";
3 import { Item } from "../types";
4
5 export default function App() {
6 const [items, setItems] = useState<Item[]>([
7 { id: "1", name: "Item 1" },
8 { id: "2", name: "Item 2" },
9]);
10
11 const deleteItem = (id: string) => {
12 setItems(items.filter((item) => item.id !== id));
13 };
14
15 return (
16 <View style={styles.container}>
17 <FlatList
18 data={items}
19 keyExtractor={(item) => item.id}
```

## Recap: Build Interactive, Typed List

- `FlatList` + `Button`: Efficiently render a list of items and let users delete them
- `useState`: Keep list reactive + Update UI automatically

### Assignment 1:

- Delete activities from the list
- Update activities from the list

# Recap: Build Interactive, Typed List

- TypeScript
- Add type annotations (`const deleteItem = (id: string) => {}`)
- Define `Item` type and use `useState<Item[]>`
- Organize types in a separate `types.ts` for reuse



# Recap: Build Interactive, Typed List

- Safe UI
  - Avoid hard-coded padding (`paddingTop: 100`)
  - Use `react-native-safe-area-context`
  - Use `<SafeAreaProvider>` and `<SafeAreaView>` for adaptive layout

# Assignment 1

Build the foundational structure of the Fitness Tracker App

- **Log** activities with details (type, duration, calories)
- **Display** activities in a list
- **Update** activities from the list
- **Delete** activities from the list
- **Input validation** and **state management**

Error Handling

# Error Handling

Notify users when input is invalid

- Use `Alert` API for simple, native dialogs

**Native dialog:** A popup rendered by the device OS (not by your code)

- Matches the platform's look & feel
- No manual styling or positioning needed

# Alert

Launch an alert dialog with specified title and message

---

```
Alert.alert("Alert Title", "Alert Message");
```

---

Example: If the user submits empty or whitespace-only text, show:

- Title: **Error**
- Message: **Please enter some text**

# Live Coding

my-app/App.tsx

```
1 import { useState } from "react";
2 import { TextInput, Button, Alert } from "react-native";
3 import { SafeAreaProvider, SafeAreaView } from "react-native-safe-area-co
4
5 export default function App() {
6 const [text, setText] = useState("");
7
8 return (
9 <SafeAreaProvider>
10 <SafeAreaView>
11 <TextInput
12 placeholder="Enter text"
13 value={text}
14 onChangeText={setText}
15 />
16 <Button title="Submit" />
17 </SafeAreaView>
18 </SafeAreaProvider>
19);
```

# Alert: Example

my-app/App.tsx

```
1 import { useState } from "react";
2 import { TextInput, Button, Alert, StyleSheet } from "react-native";
3 import { SafeAreaProvider, SafeAreaView } from "react-native-safe-area-co
4
5 export default function App() {
6 const [text, setText] = useState("");
7
8 const handleSubmit = () => {
9 if (!text.trim()) {
10 Alert.alert("Error", "Please enter some text");
11 }
12 };
13
14 return (
15 <SafeAreaProvider>
16 <SafeAreaView>
17 <TextInput
18 style={styles.input}
19 placeholder="Enter text"
```

# Alert

`Alert.alert` shows native dialogs for errors, warnings, or confirmations

Always validate inputs before processing:

- Empty or whitespace
- Invalid numbers
- Range limits

Official Doc: <https://reactnative.dev/docs/alert>

# Lecture Summary

- **React Native Basics**

- Build native apps with JavaScript & React
- Component-based UI (`View`, `Text`, `Button`, `TextInput`)

- **Styling & Layout**

- Flexbox for positioning
- Inline styles vs. `StyleSheet.create` (performance & organization)
- `<SafeAreaProvider>` and `<SafeAreaView>` for adaptive iOS and Android layouts



# Lecture Summary

- **Lists & Interactivity**
  - `ScrollView` vs. `FlatList` (performance & use cases)
  - State management with `useState`
  - Adding actions (e.g., delete) to list items
- **TypeScript Essentials**
  - Annotate props/state (`Item` type, `useState<Item[]>`)
  - Safer, clearer code
- **Error Handling:** `Alert.alert` for quick, native dialogs (OS-rendered, styled automatically)

# Assignment 1

Build the foundational structure of the Fitness Tracker App

- **Log** activities with details (type, duration, calories)
- **Display** activities in a list
- **Update** activities from the list
- **Delete** activities from the list
- **Input validation** and **state management**

Speaker notes